



**Министерство науки и высшего образования Российской Федерации
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«Московский государственный технический университет
имени Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)**

ФАКУЛЬТЕТ Информатика, искусственный интеллект и системы управления

КАФЕДРА Системы обработки информации и управления

ОТЧЕТ

по Лабораторной работе № 5.2

по дисциплине «**МЕТОДЫ МАШИННОГО ОБУЧЕНИЯ**»

Тема: «Обучение на основе временных различий»

Студент	ИУСИ-22М		Тавуз Мохамад
	(группа)	(подпись, дата)	(И.О. Фамилия)
Преподаватель			Ю.Е.Гапанюк
		(подпись, дата)	(И.О. Фамилия)

2026 г.

1) Введение

Обучение с подкреплением является одним из направлений машинного обучения, в котором агент обучается принимать решения на основе взаимодействия с окружающей средой. На каждом шаге агент находится в некотором состоянии, выбирает действие, получает вознаграждение и переходит в новое состояние. Цель агента состоит в выборе такой стратегии поведения, которая позволяет максимизировать суммарное ожидаемое вознаграждение.

В данной лабораторной работе рассматриваются методы обучения на основе временных различий. Такие методы позволяют обновлять оценки ценности действий непосредственно в процессе взаимодействия агента со средой, не требуя заранее известной полной модели переходов. Это отличает их от методов динамического программирования, которые используют модель среды.

Цель данной лабораторной работы — изучение и программная реализация базовых алгоритмов обучения с подкреплением на основе временных различий. В ходе работы были реализованы алгоритмы SARSA, Q-Learning и Double Q-Learning. Для проведения экспериментов была выбрана среда Taxi-v3 из библиотеки Gym, так как она имеет дискретное пространство состояний и действий и не относится к среде Frozen Lake, рассмотренной в лекционном примере.

2) Описание задания

В соответствии с заданием лабораторной работы необходимо реализовать методы обучения с подкреплением на основе временных различий для одной из сред библиотеки Gym или аналогичной библиотеки. При этом нельзя использовать среду Toy Text / Frozen Lake, рассмотренную в лекционном примере.

В данной работе была выбрана среда Taxi-v3 из библиотеки Gym. Эта среда содержит 500 дискретных состояний и 6 возможных действий, поэтому она хорошо подходит для табличной реализации алгоритмов SARSA, Q-Learning и Double Q-Learning. Агент должен

научиться управлять такси: забрать пассажира из заданной точки и доставить его в пункт назначения.

В ходе выполнения лабораторной работы необходимо:

- создать среду Taxi-v3 и вывести ее основные характеристики;
- реализовать базовый класс агента с Q-таблицей и epsilon-greedy стратегией выбора действий;
- реализовать алгоритм SARSA;
- реализовать алгоритм Q-Learning;
- реализовать алгоритм Double Q-Learning;
- выполнить обучение всех трех агентов;
- проверить качество обученных агентов на тестовых эпизодах;
- сравнить алгоритмы по времени обучения, средней награде, количеству шагов и успешности выполнения задачи;
- построить графики сравнения наград и количества шагов при обучении;
- сформировать итоговую таблицу результатов.

3) Текст программы и экранные формы с примерами выполнения

В данном разделе приведены основные этапы программной реализации алгоритмов SARSA, Q-Learning и Double Q-Learning для среды Taxi-v3. Все вычисления были выполнены в среде Google Colab с использованием языка программирования Python.

Для работы со средой обучения с подкреплением использовалась библиотека Gym. Для численных вычислений и хранения Q-таблиц была подключена библиотека numpy. Для формирования сравнительных таблиц результатов использовалась библиотека pandas, а для построения графиков обучения — matplotlib. Также были использованы модуль time

для измерения времени обучения и библиотека `tqdm` для отображения процесса выполнения циклов обучения.

Дополнительно было выполнено исправление совместимости Gym с NumPy 2.0, так как в новых версиях NumPy отсутствует тип `np.bool8`, который используется в некоторых версиях библиотеки Gym.

🔴 # Импорт необходимых библиотек для выполнения лабораторной работы

```
try:
    import gym
except ImportError:
    !pip install gym==0.26.2
    import gym

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import time
from tqdm import tqdm
from pprint import pprint

# Исправление совместимости Gym с NumPy 2.0
if not hasattr(np, "bool8"):
    np.bool8 = np.bool_

print("Библиотеки успешно импортированы")
print("Версия Gym:", gym.__version__)
print("Версия NumPy:", np.__version__)
```

```
... Gym has been unmaintained since 2022 and does not support NumPy 2.0 amongst other critical functionality.
Please upgrade to Gymnasium, the maintained drop-in replacement of Gym, or contact the authors of your software and request that they upgrade.
See the migration guide at https://gymnasium.farama.org/introduction/migration\_guide/ for additional information.
/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203: DeprecationWarning: datetime.datetime.utcnow() is deprecated and scheduled
    return datetime.datetime.utcnow().replace(tzinfo=utc)
Библиотеки успешно импортированы
Версия Gym: 0.25.2
Версия NumPy: 2.0.2
/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203: DeprecationWarning: datetime.datetime.utcnow() is deprecated and scheduled
    return datetime.datetime.utcnow().replace(tzinfo=utc)
```

Рисунок 1 — Импорт необходимых библиотек

3.1. Создание среды Taxi-v3 и вывод ее характеристик

На первом этапе была создана среда Taxi-v3 из библиотеки Gym. Данная среда моделирует задачу управления такси, в которой агент должен забрать пассажира из одной заданной точки и доставить его в пункт назначения. Среда имеет дискретное пространство состояний и дискретное пространство действий, что позволяет использовать табличные методы обучения с подкреплением.

После создания среды были выведены ее основные характеристики. Пространство состояний содержит 500 возможных состояний, а пространство действий содержит 6

возможных действий. Также был выведен диапазон наград и начальное состояние среды. Полученная информация подтверждает, что выбранная среда подходит для реализации алгоритмов SARSA, Q-Learning и Double Q-Learning.

```
▶ # Создание среды Taxi-v3 и вывод основных характеристик среды

env = gym.make("Taxi-v3")

print("Среда:", "Taxi-v3")
print("Пространство состояний:")
pprint(env.observation_space)

print("\nПространство действий:")
pprint(env.action_space)

print("\nДиапазон наград:")
pprint(env.reward_range)

print("\nКоличество состояний:", env.observation_space.n)
print("Количество действий:", env.action_space.n)

state = env.reset()
if isinstance(state, tuple):
    state = state[0]

print("\nНачальное состояние:", state)
```

Рисунок 2 — Код создания среды Taxi-v3

```
... Среда: Taxi-v3
    Пространство состояний:
    Discrete(500)

    Пространство действий:
    Discrete(6)

    Диапазон наград:
    (-inf, inf)

    Количество состояний: 500
    Количество действий: 6

    Начальное состояние: 408
```

Рисунок 3 — Результат создания среды Taxi-v3 и вывод характеристик

3.2. Создание базового класса агента

На следующем этапе был создан базовый класс агента BasicAgent, который используется как основа для реализации алгоритмов SARSA, Q-Learning и Double Q-Learning. В данном классе задаются основные параметры обучения, общие для всех рассматриваемых алгоритмов.

При инициализации агента определяется количество состояний и действий среды, создается Q-таблица размером 500×6 , а также задаются параметры обучения: скорость обучения learning rate = 0.1, коэффициент дисконтирования gamma = 0.98, начальное значение epsilon = 0.4, количество эпизодов обучения 5000 и максимальное количество шагов в одном эпизоде 200.

Также в базовом классе реализованы методы выбора действия. Для выбора действия используется epsilon-greedy стратегия: с вероятностью epsilon агент выбирает случайное действие, а в остальных случаях выбирает действие с максимальным Q-значением. В процессе обучения значение epsilon постепенно уменьшается, что позволяет агенту сначала исследовать среду, а затем чаще использовать найденную стратегию.

```
# Создание базового класса агента для алгоритмов временных различий

class BasicAgent:
    """
    Базовый класс агента для алгоритмов SARSA, Q-Learning и Double Q-Learning.
    """
    ALGO_NAME = "---"

    def __init__(self, env, eps=0.4, lr=0.1, gamma=0.98,
                 num_episodes=5000, max_steps=200):
        self.env = env

        # Размерности пространства состояний и действий
        self.nS = env.observation_space.n
        self.nA = env.action_space.n

        # Q-таблица
        self.Q = np.zeros((self.nS, self.nA))

        # Параметры обучения
        self.eps = eps
        self.initial_eps = eps
        self.lr = lr
        self.gamma = gamma
        self.num_episodes = num_episodes
        self.max_steps = max_steps

    # Постепенное уменьшение eps
```

```

# Постепенное уменьшение eps
self.eps_decay = eps / num_episodes
self.eps_threshold = 0.01

# История обучения
self.episodes_reward = []
self.episodes_steps = []

def get_state(self, state):
    """
    Получение номера состояния с учетом разных версий Gym.
    """
    if isinstance(state, tuple):
        return state[0]
    return state

def greedy(self, state):
    """
    Выбор действия с максимальным Q-значением.
    """
    return np.argmax(self.Q[state])

def make_action(self, state):
    """
    Выбор действия по epsilon-greedy стратегии.
    """
    if np.random.uniform(0, 1) < self.eps:
        return self.env.action_space.sample()
    return self.greedy(state)

def update_epsilon(self):
    """
    Постепенное уменьшение вероятности случайного действия.
    """
    if self.eps > self.eps_threshold:
        self.eps -= self.eps_decay

def print_info(self):
    """
    Вывод основной информации об агенте.
    """
    print("Алгоритм:", self.ALGO_NAME)
    print("Количество состояний:", self.nS)
    print("Количество действий:", self.nA)
    print("Learning rate:", self.lr)
    print("Gamma:", self.gamma)
    print("Начальное epsilon:", self.initial_eps)
    print("Количество эпизодов:", self.num_episodes)
    print("Максимальное количество шагов:", self.max_steps)
    print("Размер Q-таблицы:", self.Q.shape)

basic_agent = BasicAgent(env)
basic_agent.print_info()

```

Рисунок 4 — Код создания базового класса агента


```
... Алгоритм: ---  
Количество состояний: 500  
Количество действий: 6  
Learning rate: 0.1  
Gamma: 0.98  
Начальное epsilon: 0.4  
Количество эпизодов: 5000  
Максимальное количество шагов: 200  
Размер Q-таблицы: (500, 6)
```

Рисунок 5 — Результат инициализации базового агента

3.3. Реализация алгоритма SARSA

После создания базового класса агента был реализован алгоритм SARSA. Данный алгоритм относится к методам обучения на основе временных различий и является on-policy методом, так как обновление Q-значений выполняется с учетом действия, выбранного текущей стратегией агента.

В начале каждого эпизода выполняется инициализация состояния среды и выбирается первое действие по epsilon-greedy стратегии. После выполнения действия агент получает награду, переходит в новое состояние и выбирает следующее действие. Затем Q-значение обновляется по правилу SARSA с учетом текущего состояния, текущего действия, полученной награды, следующего состояния и следующего действия.

Обучение агента SARSA выполнялось в течение 5000 эпизодов. В результате обучение было завершено за 7.214331 секунды. Средняя награда за последние 100 эпизодов составила 7.33, а среднее количество шагов за последние 100 эпизодов — 13.49. Полученные результаты показывают, что агент научился выполнять задачу в среде Taxi-v3.


```

▶ # Реализация алгоритма SARSA

class SARSA_Agent(BasicAgent):
    """
    Агент, реализующий алгоритм SARSA.
    """
    ALGO_NAME = "SARSA"

    def learn(self):
        """
        Обучение агента на основе алгоритма SARSA.
        """
        self.episodes_reward = []
        self.episodes_steps = []
        self.eps = self.initial_eps

        start_time = time.time()

        for episode in tqdm(range(self.num_episodes)):
            state = self.get_state(self.env.reset())
            action = self.make_action(state)

            total_reward = 0
            steps = 0
            done = False
            truncated = False

            self.update_epsilon()

            while not (done or truncated) and steps < self.max_steps:
                step_result = self.env.step(action)

                if len(step_result) == 5:
                    next_state, reward, done, truncated, info = step_result
                else:
                    next_state, reward, done, info = step_result
                    truncated = False

                next_action = self.make_action(next_state)

                self.Q[state][action] = self.Q[state][action] + self.lr * (
                    reward + self.gamma * self.Q[next_state][next_action] - self.Q[state][action]
                )

                state = next_state
                action = next_action
                total_reward += reward
                steps += 1

            self.episodes_reward.append(total_reward)
            self.episodes_steps.append(steps)

        self.training_time = time.time() - start_time
        print("Обучение SARSA завершено")
        print("Время обучения:", self.training_time)
        print("Последнее значение epsilon:", self.eps)
        print("Средняя награда за последние 100 эпизодов:", np.mean(self.episodes_reward[-100:]))
        print("Среднее количество шагов за последние 100 эпизодов:", np.mean(self.episodes_steps[-100:]))

sarsa_agent = SARSA_Agent(env)
sarsa_agent.print_info()
sarsa_agent.learn()

```

Рисунок 6 — Код реализации алгоритма SARSA

```
... Алгоритм: SARSA
Количество состояний: 500
Количество действий: 6
Learning rate: 0.1
Gamma: 0.98
Начальное epsilon: 0.4
Количество эпизодов: 5000
Максимальное количество шагов: 200
Размер Q-таблицы: (500, 6)
100%|██████████| 5000/5000 [00:07<00:00, 693.80it/s] Обучение SARSA завершено
Время обучения: 7.214331388473511
Последнее значение epsilon: 0.009999999999962976
Средняя награда за последние 100 эпизодов: 7.33
Среднее количество шагов за последние 100 эпизодов: 13.49
```

Рисунок 7 — Результат обучения агента SARSA

3.4. Реализация алгоритма Q-Learning

После реализации алгоритма SARSA был реализован алгоритм Q-Learning. Данный алгоритм также относится к методам обучения на основе временных различий, однако в отличие от SARSA является off-policy методом. Это означает, что обновление Q-значений выполняется с учетом наилучшего возможного действия в следующем состоянии, независимо от того, какое действие фактически будет выбрано текущей стратегией.

На каждом шаге агент выбирает действие по epsilon-greedy стратегии, выполняет его в среде, получает награду и переходит в следующее состояние. После этого Q-значение для пары «состояние–действие» обновляется с использованием максимального Q-значения среди всех действий в следующем состоянии.

Обучение агента Q-Learning выполнялось в течение 5000 эпизодов. В результате обучение было завершено за 7.081882 секунды. Средняя награда за последние 100 эпизодов составила 7.28, а среднее количество шагов за последние 100 эпизодов — 13.18. Полученные результаты показывают, что алгоритм Q-Learning также успешно обучил агента выполнять задачу в среде Taxi-v3.

```

▶ # Реализация алгоритма Q-Learning

class QLearning_Agent(BasicAgent):
    """
    Агент, реализующий алгоритм Q-Learning.
    """
    ALGO_NAME = "Q-Learning"

    def learn(self):
        """
        Обучение агента на основе алгоритма Q-Learning.
        """
        self.episodes_reward = []
        self.episodes_steps = []
        self.eps = self.initial_eps

        start_time = time.time()

        for episode in tqdm(range(self.num_episodes)):
            state = self.get_state(self.env.reset())

            total_reward = 0
            steps = 0
            done = False
            truncated = False

            self.update_epsilon()

            while not (done or truncated) and steps < self.max_steps:
                action = self.make_action(state)

                step_result = self.env.step(action)

                if len(step_result) == 5:
                    next_state, reward, done, truncated, info = step_result
                else:
                    next_state, reward, done, info = step_result
                    truncated = False

                self.Q[state][action] = self.Q[state][action] + self.lr * (
                    reward + self.gamma * np.max(self.Q[next_state]) - self.Q[state][action]
                )

                state = next_state
                total_reward += reward
                steps += 1

            self.episodes_reward.append(total_reward)
            self.episodes_steps.append(steps)

        self.training_time = time.time() - start_time
        print("Обучение Q-Learning завершено")
        print("Время обучения:", self.training_time)
        print("Последнее значение epsilon:", self.eps)
        print("Средняя награда за последние 100 эпизодов:", np.mean(self.episodes_reward[-100:]))
        print("Среднее количество шагов за последние 100 эпизодов:", np.mean(self.episodes_steps[-100:]))

qlearning_agent = QLearning_Agent(env)
qlearning_agent.print_info()
qlearning_agent.learn()

```

Рисунок 8 — Код реализации алгоритма Q-Learning

```

... Алгоритм: Q-Learning
Количество состояний: 500
Количество действий: 6
Learning rate: 0.1
Gamma: 0.98
Начальное epsilon: 0.4
Количество эпизодов: 5000
Максимальное количество шагов: 200
Размер Q-таблицы: (500, 6)
100%|██████████| 5000/5000 [00:07<00:00, 706.47it/s] Обучение Q-Learning завершено
Время обучения: 7.0818822383880615
Последнее значение epsilon: 0.009999999999962976
Средняя награда за последние 100 эпизодов: 7.28
Среднее количество шагов за последние 100 эпизодов: 13.18

```

Рисунок 9 — Результат обучения агента Q-Learning

3.5. Реализация алгоритма Double Q-Learning

После реализации SARSA и Q-Learning был реализован алгоритм Double Q-Learning. Данный алгоритм является модификацией Q-Learning и предназначен для уменьшения переоценки Q-значений, которая может возникать при использовании максимума по действиям в классическом Q-Learning.

В Double Q-Learning используются две отдельные Q-таблицы: Q1 и Q2. На каждом шаге с вероятностью 0.5 обновляется одна из этих таблиц. Если обновляется Q1, то лучшее действие в следующем состоянии выбирается по Q1, а его значение оценивается по Q2. Если обновляется Q2, то действие выбирается по Q2, а значение оценивается по Q1. Для выбора действия во время взаимодействия со средой используется сумма двух таблиц Q1 + Q2.

Обучение агента Double Q-Learning выполнялось в течение 5000 эпизодов. В результате обучение было завершено за 8.981478 секунды. Средняя награда за последние 100 эпизодов составила 7.87, а среднее количество шагов за последние 100 эпизодов — 12.86. Также были сформированы две Q-таблицы размером 500×6 , что соответствует количеству состояний и действий среды Taxi-v3.

```

▶ # Реализация алгоритма Double Q-Learning

class DoubleQLearning_Agent(BasicAgent):
    """
    Агент, реализующий алгоритм Double Q-Learning.
    """
    ALGO_NAME = "Double Q-Learning"

    def __init__(self, env, eps=0.4, lr=0.1, gamma=0.98,
                 num_episodes=5000, max_steps=200):
        super().__init__(env, eps, lr, gamma, num_episodes, max_steps)

        # В Double Q-Learning используются две Q-таблицы
        self.Q1 = np.zeros((self.nS, self.nA))
        self.Q2 = np.zeros((self.nS, self.nA))

    def greedy(self, state):
        """
        Выбор действия по сумме двух Q-таблиц.
        """
        return np.argmax(self.Q1[state] + self.Q2[state])

    def make_action(self, state):
        """
        Выбор действия по epsilon-greedy стратегии.
        """
        if np.random.uniform(0, 1) < self.eps:
            return self.env.action_space.sample()
        return self.greedy(state)

    def learn(self):
        """
        Обучение агента на основе алгоритма Double Q-Learning.
        """
        self.episodes_reward = []
        self.episodes_steps = []
        self.eps = self.initial_eps

        start_time = time.time()

        for episode in tqdm(range(self.num_episodes)):
            state = self.get_state(self.env.reset())

            total_reward = 0
            steps = 0
            done = False
            truncated = False

            self.update_epsilon()

            while not (done or truncated) and steps < self.max_steps:
                action = self.make_action(state)

                step_result = self.env.step(action)

                if len(step_result) == 5:
                    next_state, reward, done, truncated, info = step_result
                else:
                    next_state, reward, done, info = step_result
                    truncated = False

                if np.random.rand() < 0.5:
                    best_next_action = np.argmax(self.Q1[next_state])
                    self.Q1[state][action] = self.Q1[state][action] + self.lr * (
                        reward + self.gamma * self.Q2[next_state][best_next_action] - self.Q1[state][action]
                    )
                else:
                    best_next_action = np.argmax(self.Q2[next_state])
                    self.Q2[state][action] = self.Q2[state][action] + self.lr * (
                        reward + self.gamma * self.Q1[next_state][best_next_action] - self.Q2[state][action]
                    )

                state = next_state
                total_reward += reward
                steps += 1

            self.episodes_reward.append(total_reward)
            self.episodes_steps.append(steps)

        # Итоговая Q-таблица используется для проверки агента
        self.Q = self.Q1 + self.Q2
        self.training_time = time.time() - start_time

        print("Обучение Double Q-Learning завершено")
        print("Время обучения:", self.training_time)
        print("Последнее значение epsilon:", self.eps)
        print("Средняя награда за последние 100 эпизодов:", np.mean(self.episodes_reward[-100:]))
        print("Среднее количество шагов за последние 100 эпизодов:", np.mean(self.episodes_steps[-100:]))
        print("Размер Q1-таблицы:", self.Q1.shape)
        print("Размер Q2-таблицы:", self.Q2.shape)

double_q_agent = DoubleQLearning_Agent(env)
double_q_agent.print_info()
double_q_agent.learn()

```

Рисунок 10 — Код реализации алгоритма Double Q-Learning

```

... Алгоритм: Double Q-Learning
Количество состояний: 500
Количество действий: 6
Learning rate: 0.1
Gamma: 0.98
Начальное epsilon: 0.4
Количество эпизодов: 5000
Максимальное количество шагов: 200
Размер Q-таблицы: (500, 6)
100%|██████████| 5000/5000 [00:08<00:00, 557.01it/s] Обучение Double Q-Learning завершено
Время обучения: 8.981478452682495
Последнее значение epsilon: 0.0099999999999962976
Средняя награда за последние 100 эпизодов: 7.87
Среднее количество шагов за последние 100 эпизодов: 12.86
Размер Q1-таблицы: (500, 6)
Размер Q2-таблицы: (500, 6)

```

Рисунок 11 — Результат обучения агента Double Q-Learning

3.6. Проверка обученных агентов

После завершения обучения всех трех агентов была выполнена проверка их работы на тестовых эпизодах. На данном этапе агенты больше не выполняли случайные действия, а выбирали действия жадно, то есть в соответствии с максимальными значениями в обученных Q-таблицах.

Для каждого алгоритма было выполнено 20 тестовых эпизодов. В каждом эпизоде подсчитывались суммарная награда, количество шагов до завершения задачи и признак успешного выполнения. Такой подход позволяет оценить качество уже обученной стратегии отдельно от процесса исследования среды во время обучения.

Результаты проверки показали, что все три алгоритма успешно завершили 20 из 20 тестовых эпизодов. Для SARSA средняя тестовая награда составила 7.35, а среднее количество шагов — 13.65. Для Q-Learning средняя тестовая награда составила 8.15, а среднее количество шагов — 12.85. Для Double Q-Learning средняя тестовая награда составила 7.40, а среднее количество шагов — 13.60.


```

▶ # Проверка обученных агентов после завершения обучения

def evaluate_agent(env, agent, episodes_count=20, max_steps=200):
    """
    Проверка качества обученного агента на нескольких тестовых эпизодах.
    """
    rewards = []
    steps_list = []
    successful_episodes = 0

    for episode in range(episodes_count):
        state = agent.get_state(env.reset())

        total_reward = 0
        steps = 0
        done = False
        truncated = False

        while not (done or truncated) and steps < max_steps:
            action = agent.greedy(state)
            step_result = env.step(action)

            if len(step_result) == 5:
                next_state, reward, done, truncated, info = step_result
            else:
                next_state, reward, done, info = step_result
                truncated = False

            total_reward += reward
            state = next_state
            steps += 1

            if total_reward > 0:
                successful_episodes += 1

            rewards.append(total_reward)
            steps_list.append(steps)

    return rewards, steps_list, successful_episodes

sarsa_test_rewards, sarsa_test_steps, sarsa_successful = evaluate_agent(env, sarsa_agent)
qlearning_test_rewards, qlearning_test_steps, qlearning_successful = evaluate_agent(env, qlearning_agent)
double_q_test_rewards, double_q_test_steps, double_q_successful = evaluate_agent(env, double_q_agent)

print("Проверка обученных агентов завершена")

print("\nSARSA:")
print("Количество успешных эпизодов:", sarsa_successful)
print("Средняя награда:", np.mean(sarsa_test_rewards))
print("Среднее количество шагов:", np.mean(sarsa_test_steps))

print("\nQ-Learning:")
print("Количество успешных эпизодов:", qlearning_successful)
print("Средняя награда:", np.mean(qlearning_test_rewards))
print("Среднее количество шагов:", np.mean(qlearning_test_steps))

print("\nDouble Q-Learning:")
print("Количество успешных эпизодов:", double_q_successful)
print("Средняя награда:", np.mean(double_q_test_rewards))
print("Среднее количество шагов:", np.mean(double_q_test_steps))

```

Рисунок 12 — Код проверки обученных агентов

... Проверка обученных агентов завершена

SARSA:

Количество успешных эпизодов: 20

Средняя награда: 7.35

Среднее количество шагов: 13.65

Q-Learning:

Количество успешных эпизодов: 20

Средняя награда: 8.15

Среднее количество шагов: 12.85

Double Q-Learning:

Количество успешных эпизодов: 20

Средняя награда: 7.4

Среднее количество шагов: 13.6

Рисунок 13 — Результат проверки обученных агентов

3.7. Формирование сравнительной таблицы результатов алгоритмов

После проверки обученных агентов была сформирована сравнительная таблица результатов алгоритмов SARSA, Q-Learning и Double Q-Learning. В таблицу были включены основные показатели качества обучения и тестирования: время обучения, средняя награда за последние 100 эпизодов обучения, среднее число шагов при обучении, количество успешных тестовых эпизодов, средняя тестовая награда и среднее число шагов при тестировании.

Сравнительная таблица показывает, что все три алгоритма успешно решили задачу в среде Taxi-v3 и завершили 20 из 20 тестовых эпизодов. По средней тестовой награде лучший результат показал алгоритм Q-Learning: средняя награда составила 8.15. Этот же алгоритм показал минимальное среднее количество шагов при тестировании — 12.85.

Алгоритм SARSA получил среднюю тестовую награду 7.35 при среднем количестве шагов 13.65. Алгоритм Double Q-Learning получил среднюю тестовую награду 7.40 при среднем количестве шагов 13.60. Таким образом, в данном эксперименте Q-Learning оказался наиболее эффективным среди реализованных методов.



Формирование сравнительной таблицы результатов алгоритмов

```
comparison_table = pd.DataFrame({
    "Алгоритм": [
        "SARSA",
        "Q-Learning",
        "Double Q-Learning"
    ],
    "Время обучения": [
        sarsa_agent.training_time,
        qlearning_agent.training_time,
        double_q_agent.training_time
    ],
    "Средняя награда при обучении": [
        np.mean(sarsa_agent.episodes_reward[-100:]),
        np.mean(qlearning_agent.episodes_reward[-100:]),
        np.mean(double_q_agent.episodes_reward[-100:])
    ],
    "Среднее число шагов при обучении": [
        np.mean(sarsa_agent.episodes_steps[-100:]),
        np.mean(qlearning_agent.episodes_steps[-100:]),
        np.mean(double_q_agent.episodes_steps[-100:])
    ],
    "Успешных тестовых эпизодов": [
        sarsa_successful,
        qlearning_successful,
        double_q_successful
    ],
    "Средняя тестовая награда": [
        np.mean(sarsa_test_rewards),
        np.mean(qlearning_test_rewards),
        np.mean(double_q_test_rewards)
    ],
    "Среднее число шагов при тестировании": [
        np.mean(sarsa_test_steps),
        np.mean(qlearning_test_steps),
        np.mean(double_q_test_steps)
    ]
})

comparison_table
```

Рисунок 14 — Код формирования сравнительной таблицы результатов

	Алгоритм	Время обучения	Средняя награда при обучении	Среднее число шагов при обучении	Успешных тестовых эпизодов	Средняя тестовая награда	Среднее число шагов при тестировании
0	SARSA	7.214331	7.33	13.49	20	7.35	13.65
1	Q-Learning	7.081882	7.28	13.18	20	8.15	12.85
2	Double Q-Learning	8.981478	7.87	12.86	20	7.40	13.60

Рисунок 15 — Сравнительная таблица результатов алгоритмов

3.8. Построение графика сравнения наград при обучении

Для наглядного анализа процесса обучения был построен график сравнения наград, получаемых агентами в ходе обучения. Так как значения наград по отдельным эпизодам могут сильно колебаться, для отображения использовалось скользящее среднее за последние 100 эпизодов.

На графике представлены результаты трех алгоритмов: SARSA, Q-Learning и Double Q-Learning. По оси абсцисс отложен номер эпизода, а по оси ординат — средняя награда за последние 100 эпизодов. В начале обучения значения наград являются отрицательными, так как агенты еще активно исследуют среду и часто совершают неэффективные действия.

По мере увеличения количества эпизодов средняя награда постепенно возрастает, что свидетельствует об улучшении стратегий поведения агентов. К концу обучения все три алгоритма показывают положительные значения средней награды. При этом график подтверждает, что реализованные методы временных различий позволяют агентам обучиться эффективному поведению в среде Taxi-v3.

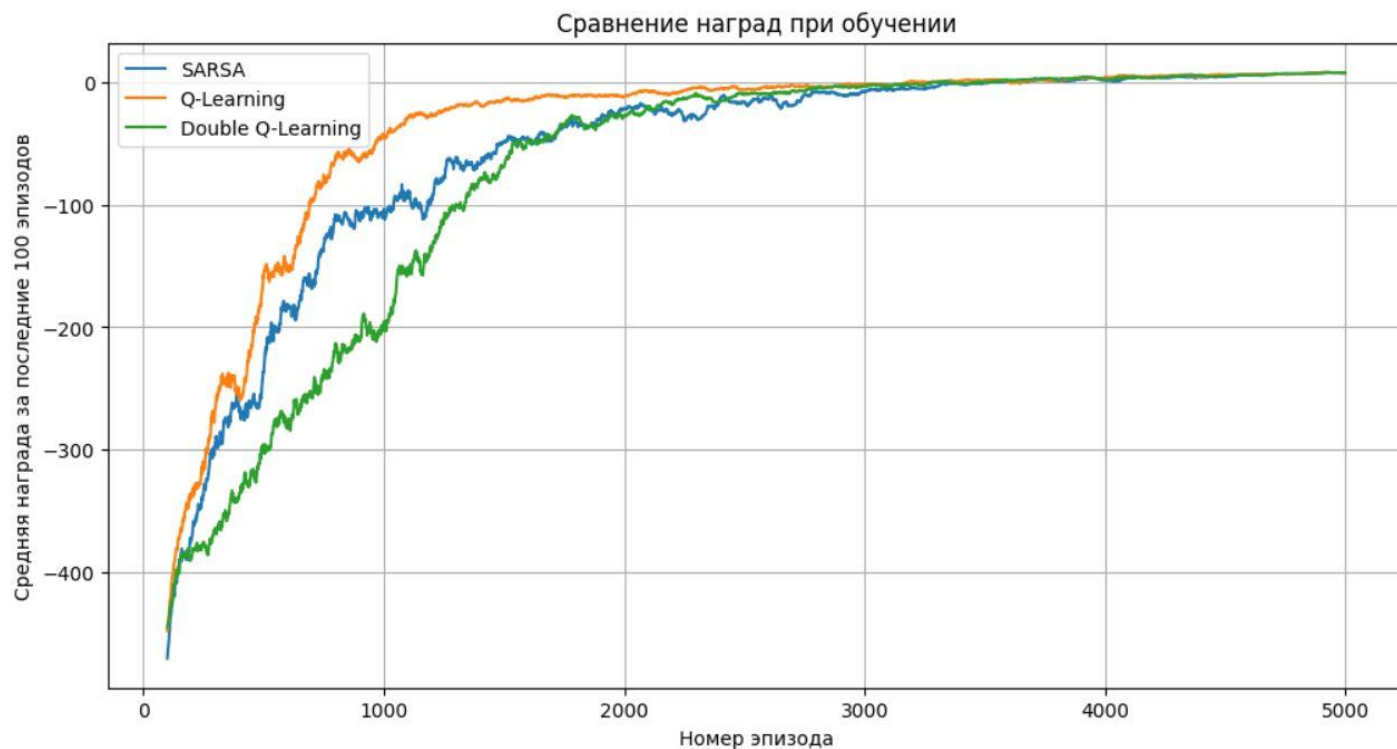


Рисунок 16 — График сравнения наград при обучении

3.9. Построение графика сравнения количества шагов при обучении

Дополнительно был построен график сравнения количества шагов, необходимых агентам для завершения эпизодов в процессе обучения. Как и при анализе наград, для сглаживания случайных колебаний использовалось скользящее среднее за последние 100 эпизодов.

На графике представлены результаты алгоритмов SARSA, Q-Learning и Double Q-Learning. По оси абсцисс отложен номер эпизода, а по оси ординат — среднее количество шагов за последние 100 эпизодов. В начале обучения агенты часто совершают лишние действия, поэтому количество шагов является большим.

В процессе обучения среднее количество шагов постепенно уменьшается. Это означает, что агенты находят более эффективные последовательности действий для перевозки пассажира в среде Taxi-v3. К концу обучения все три алгоритма показывают близкие результаты, что подтверждает успешное формирование эффективных стратегий поведения.



Рисунок 17 — График сравнения количества шагов при обучении

3.10. Формирование итоговой таблицы лабораторной работы

На заключительном этапе выполнения программы была сформирована итоговая таблица лабораторной работы. В нее были включены основные характеристики проведенного эксперимента: выбранная среда, количество состояний, количество действий, количество эпизодов обучения, максимальное количество шагов, лучший алгоритм по средней тестовой награде, лучшая средняя тестовая награда, минимальное среднее число шагов при тестировании и количество успешных тестовых эпизодов.

Итоговая таблица показывает, что для проведения экспериментов была использована среда Taxi-v3, содержащая 500 состояний и 6 действий. Обучение каждого агента выполнялось в течение 5000 эпизодов, при этом максимальное количество шагов в одном эпизоде было ограничено значением 200.

По результатам тестирования лучшим алгоритмом оказался Q-Learning. Он показал наибольшую среднюю тестовую награду 8.15 и минимальное среднее число шагов при тестировании 12.85. Все три реализованных алгоритма успешно завершили 20 из 20

тестовых эпизодов, что подтверждает корректность реализации методов временных различий.

	Показатель	Значение
0	Среда	Taxi-v3
1	Количество состояний	500
2	Количество действий	6
3	Количество эпизодов обучения	5000
4	Максимальное количество шагов	200
5	Лучший алгоритм по средней тестовой награде	Q-Learning
6	Лучшая средняя тестовая награда	8.15
7	Минимальное среднее число шагов при тестировании	12.85
8	Количество успешных тестовых эпизодов для кажд...	20 из 20

Рисунок 18 — Итоговая таблица лабораторной работы

4) Заключение

В ходе лабораторной работы были изучены и реализованы базовые методы обучения с подкреплением на основе временных различий. В качестве среды для проведения экспериментов была выбрана Taxi-v3 из библиотеки Gym. Данная среда имеет дискретное пространство состояний и действий, поэтому она подходит для табличной реализации алгоритмов SARSA, Q-Learning и Double Q-Learning.

В процессе выполнения работы был создан базовый класс агента, реализована epsilon-greedy стратегия выбора действий, а также выполнена программная реализация трех алгоритмов: SARSA, Q-Learning и Double Q-Learning. Каждый агент обучался в течение 5000 эпизодов, после чего была проведена проверка качества полученных стратегий на 20 тестовых эпизодах.

Результаты тестирования показали, что все три алгоритма успешно завершили 20 из 20 тестовых эпизодов. Алгоритм SARSA получил среднюю тестовую награду 7.35, алгоритм Q-Learning — 8.15, а алгоритм Double Q-Learning — 7.40. Минимальное среднее количество шагов при тестировании также показал алгоритм Q-Learning — 12.85 шага.

Таким образом, все поставленные задачи лабораторной работы были выполнены. Реализованные алгоритмы временных различий позволили обучить агентов эффективному поведению в среде Taxi-v3. По результатам проведенного эксперимента наиболее эффективным оказался алгоритм Q-Learning, который показал наибольшую среднюю тестовую награду и наименьшее среднее количество шагов при тестировании.